UNIVERSITY OF LIVERPOOL

Department of Electrical Engineering and Electronics

ELEC473

Digital Systems Design
Assignment 4 Resit
NIOS II

Module	ELEC473
Coursework name	Assignment 4
Component weight	20%
Semester	2
HE Level	7
Lab location	N/A
Work	Individually
Timetabled time	N/A
Suggested private study	10 hours including report writing
Assessment method	Individual, formal word-processed reports (Block diagrams and ASMs can be hand drawn and scanned into the report)
Submission format	Online via CANVAS
Submission deadline	23:59 on Friday 31st July 2026
Late submission	Standard university penalty applies
Resit opportunity	N/A
Marking policy	Marked and moderated independently
Anonymous marking	Yes
Feedback	Via comments on Canvas
Learning outcomes	LO1: Ability to design digital systems using the ASM design method LO2: Ability to implement digital systems using the Verilog Hardware Description Language LO4: Ability to implement a SOPC system using Quartus Nios-II.

Marking Criteria

Section	Marks available	Indicative characteristics	
		Adequate / pass (50%)	**Very good / Excellent**
Presentation and structure	20%	• Contains cover page information, table of contents, sections with appropriate headings. • Comprehensible language; punctuation, grammar and spelling accurate. • Equations legible, numbered and presented correctly. • Appropriately formatted reference list.	• Appropriate use of technical, mathematic and academic terminology and conventions. • Word processed with consistent formatting. • Pages numbered, figures and tables captioned. • All sections clearly signposted. • Correct cross-referencing (of figures, tables, equations) and citations.
Introduction, Method and Design	40%	• Problem background introduced clearly. • Evidence of a Top Down Design approach • Conceptual Design Choices introduced. • Design of each module follows a logical sequence. • ASMs correspond to designs for each block. • Software is clearly commented	• Appropriate range of references used. • Design decisions justified with alternatives given. • Calculations shown in full, justifying and explaining any decisions. • Correct ASM Syntax used. • Well-structured Verilog Code
Results	30%	• Simulation results present for each block and well annotated. • Results of full system in both simulation and experimentally presented. • Results for each task accompanied by a commentary. • Screen shots of results presented.	• Tests indicate that there are no problems caused by asynchronous inputs. • Clear explanation of how the instructions operate correctly
Discussion	10%	• Discussion on what worked and what didn't. • Critical assessment on the design – strength and weaknesses	• Discussion on how the system was fully tested.

ELEC 473 Assignment 4 (2025-26) Synthesising the NIOS II Processor

Assignment Outline

In Assignment 3 you added some extra instructions to a MIPS processor which was then synthesised and executed on the DE2 board. This assignment aims to introduce you to a commercial synthesised processor targeted for Altera FPGAs, which allows the easy importing of peripherals and the use of an industry standard IDE for software development.

Assignment 4 is split into three parts, Parts A, B and C. The objective of Part A is to get you familiar with QSYS, the NIOS II Processor and interfacing SRAM and SDRAM, on the Altera DE2 Board, to the NIOS II processor you synthesise. Part B requires you to develop and test a Custom Instruction that will be implemented in the FPGA. Part C requires you to generate a PWM output.

Part A – SRAM & SDRAM

The Nios II hardware development tutorial synthesises a NIOS II processor with, 20 KB of on-chip memory, a timer, a JTAG UART, 8 parallel I/O pins and a system ID Component. Part A of assignment 4 requires you to interface the 512 KB SRAM and 8 MB SDRAM on the Altera DE2 Board to this design.

You should then test that the memory is functioning correctly by running the Memory Test programs available within the NIOS II IDE. *You should modify the Memory Test Programme so that your Name and ID number are shown in the terminal window each time the memory is tested.*

Hints

1. Initially use the altera_up_avalon_sram Controller (SRAM/SSRAM) for the SRAM interface
2. Use the SDRAM controller for the SDRAM Interface
3. Add a PLL to advance the clock for the SDRAM by 3ns compared with the system clock.

Part B – Custom Instruction

Part B requires you to develop a Custom Instruction to count the number of leading 1s (or 0s – see Table 1) in the 32 bit number passed to the instruction (for example 0xFF000000 would have 8 leading 1s and 0 leading 0s, whilst 0x00F00000 would have 0 leading 1s and 8 leading 0s).. You should write a program to test your Custom Instruction. You should also develop a test routine in C or assembler that performs the same function as the Custom Instruction and compare the speed of the Custom Instruction against your software implementation.

Part C – PWM module

Pulse Width Modulation (PWM) has many uses, but it main application is to allow the control of the power to electrical devices. For Part C, you are to use PWM to control the intensity of an LED (See Table 1) on the DE2 Board. You should write software so that the intensity gradually varies from off to fully on to fully off and then repeats.

To implement a PWM generator typically three components are required a) A PWM register which the programme writes to, that stores the required PWM value, b) A counter, of the same width as the PWM register, that counts up to the maximum value, goes back to zero and then repeats, c) A comparator that compares the value of the counter with that of the register and then generates the single bit PWM output.

There are many ways that you can implement the PWM module, the simplest way is to use a QSYS GPIO module as the register that the programme can write to and then outside QSYS generate the counter and comparator. A more elegant solution is to generate a full QSYS block that has the register and comparator and single bit output.

Submission

You report should include the following:

1. Block diagram of the BDF developed in Part A.
2. Table showing memory Map for Part A
3. Screen bumps of test results showing memory test programs working for SRAM and SDRAM
4. ASM(s) and Verilog code for your custom instruction in Part B
5. C/C++ for your test program of Part B
6. Screen dump showing the results of your program for Part B
7. Results showing a speed comparison between the Custom Instruction and your software implementation.
8. Explanation of your results for Part B.
9. Block diagram and schematic of your design for Part C.
10. ASM Charts for your design for Part C.
11. Verilog code for your design in Part C.
12. Software design for Part C.
13. Explanation of results for Part C.

Submission Deadline

Electronic copy: Friday 31st July 2026 @ 11:59pm

Table 1 Part B Custom Instruction Task Part C PWM LED

ID	Name	Counting	PWM LED
201926897	Abdelfattah, Laila Maged Mahmoud	Leading 0's	LEDR1
201954749	McKee, Ryan	Leading 1's	LEDR10
201581462	Mehany, Youssef Hussein Abdelghany	Leading 0's	LEDR2
201619433	Sheikh, Rameez Shafi	Leading 1's	LEDR14